

Major Design and Development Considerations

- The main considerations in this program was how to most effectively split the classes and methods up. I never wanted to contain too many methods in one class to make it feel clunky. I found it very useful to create a class, “Die”, and another class, “Hand”. This allowed me to create Array Lists full of the object, “Die”, and use methods in “Hand” to create a hand, and show it to the user. Although, in the future, I could have implemented this more effectively. Another major development consideration was whether to make variables static or not. For example, in “Die”, I decided not to make faceValue static. This made it more difficult to access it in an ArrayList, but at the beginning I wanted to practice not having it seen by the whole program.

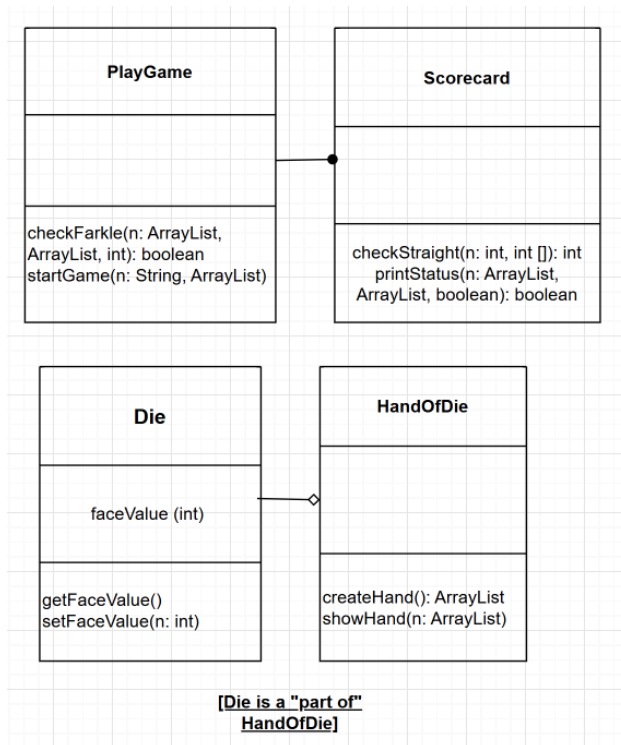
Goal Of My Program

- My program implementation, “Farkle”, models the real game that includes elements such as dice, the hand, scorecards, and melds. It allows single-players to log on and play the game. Right now, there is only one turn implemented at a time where they can roll the dice, see what they rolled, and decide whether or not they want to add to their meld, once the machine tells them if they have a farkle or not. At the end, they get the total score of that round.

Design of the Program

- Due to Java’s Object-Oriented style of programming, I found it helpful to separate important methods, and variables into several different classes. The Farkle class contains a main method that starts the program. It initiates the hand, and then sends the user into the PlayGame class. This is where the bulk of the code lies to deal with Farkles, Melds and scoring. I also decided to make two different classes, “Die”, and “HandOfDie”, to keep these objects separate. I first started as initiating a hand as an array, but soon found it was better suited to be an arrayList. My program leads the user efficiently through this board game through the readable user interface and always reminding them what dice they have. At the end, the total score is provided, and they can always play again if they want! (just will have to run the program a second time).

A UML Class Diagram



- Did not include Farkle: only the main method

Design and programming Issues

- One of the main issues that I came upon was dusting the rust off of my Java skills. In the back of my mind, there were several methods that came to mind to make some coding easier, but I had to refresh my mind on what they specifically looked like. Before I could figure out some more basics, this stopped me in my tracks. I am excited to code more like this when we learn more in class.
- Another issue was dealing with the mess I created for myself in the beginning. I accidentally implemented the DrewFarkle implementation provided, but realized I needed to change it to the Crandall version. This ended up helping me in the end because Crandall seemed to be more intuitive. Instead of getting frustrated easily, I took a step back and saw what I needed to do to further my progress. I got rid of specific methods, and re-oriented my code.
- Finally, figuring out what to include as arguments in each of the methods came up as a challenge. I kept switching certain objects around, such as what ArrayList to pass in. This was a major issue because the static inspector had errors come up constantly. However, I

could catch these failures early, and I addressed them by backtracking and looking at my code with a bird's eye view.

What would I have done differently

- I understand that the assignment stated to have my implementation look almost identical to the C++ version, but in my opinion, this implementation could have been sturdier and easier to read. Next time, I would be excited to code this from scratch. Another improvement I would make is how I decided to design the checkStraight portion, combined with the checkStatus method. Next time, I would consider putting these both into their own classes, or in a "Total" class to separate them out. Finally, I would consider designing my program in a new way in the fact that I did not have many attributes in each of my classes. I did not realize this until my UML diagram, but this is definitely something that I could have benefitted from using the Java language.